



POLITECHNIKA ŚLĄSKA
Wydział Inżynierii Materiałowej
Rok akademicki: 2024/25

Sieci komputerowe

TEMAT:

Projekt sieciowy symulator UAR

Uwagi prowadzącego:

Data przyjęcia:

Podpis prowadzącego:

Imię i nazwisko:

- 1. Konrad Szkółka**
- 2. Michał Witczak**

Grupa dziekańska: 2

Sekcja laboratoryjna: 2

1) Podział prac:

a) Michał Witczak:

- Stworzenie dwóch klas TCPKlient, TCPServer do komunikacji sieciowej pomiędzy instancjami.
- Testowanie poprawności przesyłu danych pomiędzy instancjami
- Refaktoryzacja komunikacji sieciowej – finalnie powstała tylko jedna klasa odgrywająca obie role (serwer i klient)
- Naprawienie “migającej lampki”
- Dodanie rysowanie wykresów po brakującej stronie (serwer)
- Testowanie przesyłu poprawnej konfiguracji przy odłączeniu jednej instancji od drugiej
- Komunikacja w zespole jak i z poprzednimi właścicielami kodu
- Finalne testowanie programu

b) Konrad Szkółka:

- Serializacja i deserializacja przesyłu danych
- Testowanie nowej komunikacji
- Modyfikacje GUI
- Dodanie rysowanie wykresów po stronie klienta
- Naprawienie problemów z nieprawidłowymi wartościami na wykresach
- Stworzenie zapisu konfiguracji modelu i regulatora przy odłączeniu jednej instancji od drugiej
- Konsultacje z sekcją odpowiedzialną za podstawowy projekt
- Finalne testowanie programu

2) Historia rozwoju aplikacji:

- a) Dodano dwie klasy TCPClient, TCPServer i ich wstępne funkcjonalności. W postaci sprawdzenia czy dwie instancje się widzą.
- b) Dodanie do GUI odpowiednich narzędzi - takich jak: button, label, textedit, radiobutton
- c) Rozbudowano klasy o serializację i deserializację danych - na ten moment przesyłu jednego double'a i dwóch int-ów.
- d) Dodanie do GUI lampki po obu stronach informującej o połączeniu serwer <-> klient
- e) Dodanie powiadomienia o nieotrzymanych ramach danych w postaci świecącej lampki na czerwono i wdrożenie zapobiegawczych metod, takich jak wywołanie pewnych i potrzebnych obliczeń lokalnie.
- f) Rozbudowa przesyłu danych do finalnej postaci, czyli do wysyłania i odbierania wszystkich potrzebnych danych. Ramka jest tworzona w postaci struktury danych i przesyłana za pomocą: QByteArray + QDataStream.

- g) Dodanie w GUI blokady na zmianę wartości w okienkach, gdy instancja jest odpowiedziem serverem czy klientem
- h) Zmienie koncepcji komunikacji klienta z serverem poprzez refaktoryzację. Klas odpowiedzialnych za ów komunikację. Złączenie TCPClient i TCPServer do klasy MyServer.
- i) Dodanie automatycznej zmiany konfiguracji po obu stronach jak tylko, zostanie zmieniona wartość. Dla przykładu gdy klient zmienia wartość "A" dla ARX to po otrzymaniu ramki przez server ona się automatycznie zapisze jako nowa wartość dla "A" w ARX. Co pozwala po zerwaniu połączenia lub utracie ramki przy działaniu tego połączenia na poprawne obliczenie i wyświetlenie na wykres

3) Napotkane błędy i uzasadnienie refaktoryzacji:

Problem:

Komunikacja była rozwiązana w sposób teoretyczny, czyli taki, że jak np.: server sprawdzał, czy jest połączenie to jak był to wyświetlał komunikat w label "połączono" lub "brak połączenia"

Rozwiązanie:

Dodanie dodatkowej lampki jak i dodanie komunikatu do label w postaci IPklienta i jaki po jakim porcie się łączy dla serwera, a dla klioenta IPserwera i port serwera

Problem:

Lampka nie chciała się zaświecić na zielono po stronie klienta pomimo widocznej komunikacji w postaci, otrzymania i wysyłania ramek z powrotem. Na serwerze nie było tego problemu.

Rozwiązanie:

Dodanie wymuszenia świecenia się lampki na zielono do momentu braku otrzymania ramki poprzez wykorzystanie QTimer. Wtedy zmieniał się kolor lampki na czerwony

Problem:

Nierozwiązalny na tamtą chwilę i naszą wiedzę na temat programowania, było gubienie ramek pomiędzy klientem -> serverem. Server prawidłowo wysyłał dane, klient je odbierał jak i obliczał na ich podstawie wyjście z ARX natomiast nie mógł wysłać ramek a server jej odebrać.

Rozwiązanie:

Z racji niemożności znalezieniu przyczyny jak i zbliżającemu się nieubłagane terminowi oddania. Podjeliśmy wspólną decyzję o zastąpieniu klas odpowiedzialnymi za komunikację na jedną większą, ale z automatycznym wykryciem czy program ma być serwerem czy klientem. Zrezygnowaliśmy z dużej ilości funkcji a postawiliśmy bardziej na funkcjonalności "emit"

4) Czego się nauczyliśmy:

a) Michał Witczak:

- Zarządzania komunikacją sieciową w QT opartiu na QTcpSocket QTcpSerwer
- Używania QDebuq aby lepiej znaleźć błędy w kodzie
- Nauczeniu czytania i rozumieniu pozornie “łatwego” kodu programu
- Polepszenie wiedzy na temat właściwości i możliwości QT

b) Konrad Szkółka

- Wykrywania utraty połączenia jak i braku dostarczenia ramek danych za pomocą QTimer
- Serializacji i deserializacji z wykorzystaniem QByteArray + QDataStream
- Powiększenie wiedzy na temat samego działania QT jak i GUI i rysowania wykresów